

ヒープソート

「基本3種」や「クイック・マージ」の次に登場する、少し高度なソートが「**ヒープソート (Heap Sort)**」です。

基本情報技術者試験では、「木構造 (ツリー)」という概念と一緒に登場するため難しく感じがちですが、本質は「**トーナメント形式で一番強い (大きい) やつを効率よく見つける**」という非常に合理的な仕組みです。

1. ヒープソートの核心：ヒープ構造

ヒープソートを理解するには、まず「**ヒープ**」というデータの持ち方を知る必要があります。

- **ヒープとは**：「親は子よりも必ず大きい (または小さい)」というルールで並んだピラミッド状のネットワーク (木構造) のことです。
 - **最大の特徴**：このルールを守ると、**一番てっぺん (根) には必ず「最大値」が来る**ようになります。
-

2. 具体的な手順

基本情報技術者試験では、「**配列で表現されたヒープ**」を読み解く問題がよく出ます。

「インデックス0番がルート、1番と2番がその子、3番と4番は1番の子...」といった配列の並びと、ツリー構造を頭の中で一致させられるかが鍵になります。

ここでは、「**配列で表現されたヒープ**」を使ったヒープソートの例を挙げます。

大きく分けて2つのフェーズで進みます。

フェーズ①：ヒープを作る

例として、[4, 10, 3, 5, 1] という配列を使います。

下図で、**0:4** は配列のインデックス**0**の箱に**4**という値が入っているという意味です。

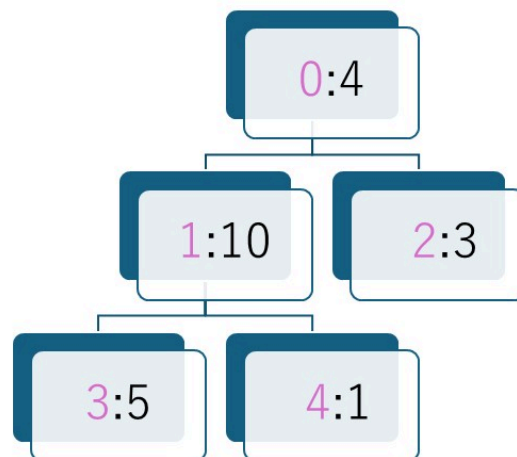
![note]

ここでは、配列のインデックスは0始まりとしています。疑似言語を想定する場合は、1始まりに読み替えてください。

Array

0	1	2	3	4
4	10	3	5	1

Heap



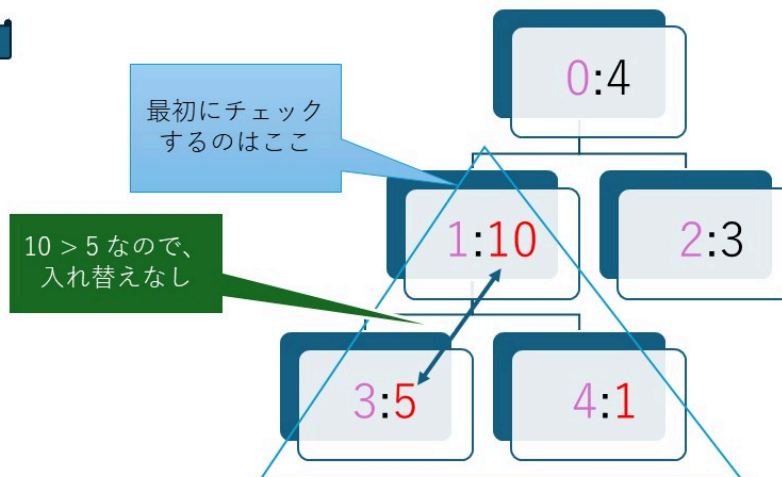
バラバラに並んだデータを、まず「親が子より大きい」というヒープの状態に並べ替えます。これで、**ピラミッドの頂点に最大値** がやってきます。

- まず、^{リーフ}葉（一番下のノード）を除く親ノードに対して、下から順にヒープ化を実行します
- 左ノード → 右ノード の順に、「親 > 子」のルールに則っているか、調べます。

Array

0	1	2	3	4
4	10	3	5	1

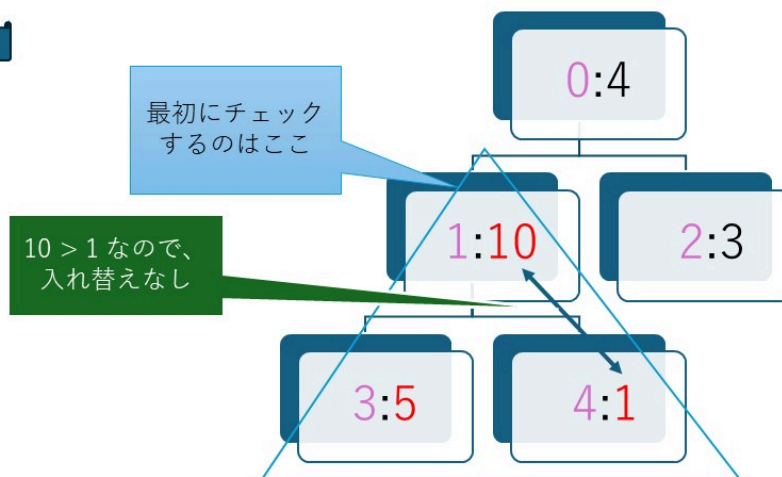
Heap



Array

0	1	2	3	4
4	10	3	5	1

Heap

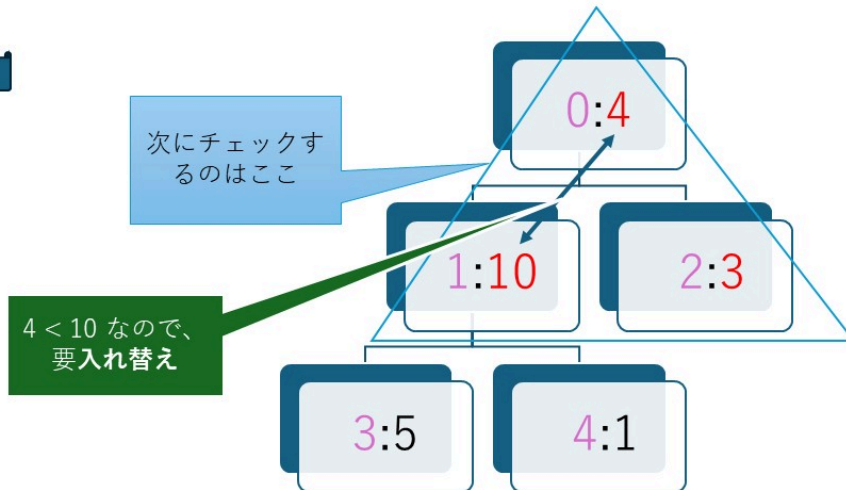


- 最初の分木がヒープ化されたので、次の分木に移ります
- 「親 > 子」のルールに則っていないものは入れ替えます。

Array

0	1	2	3	4
4	10	3	5	1

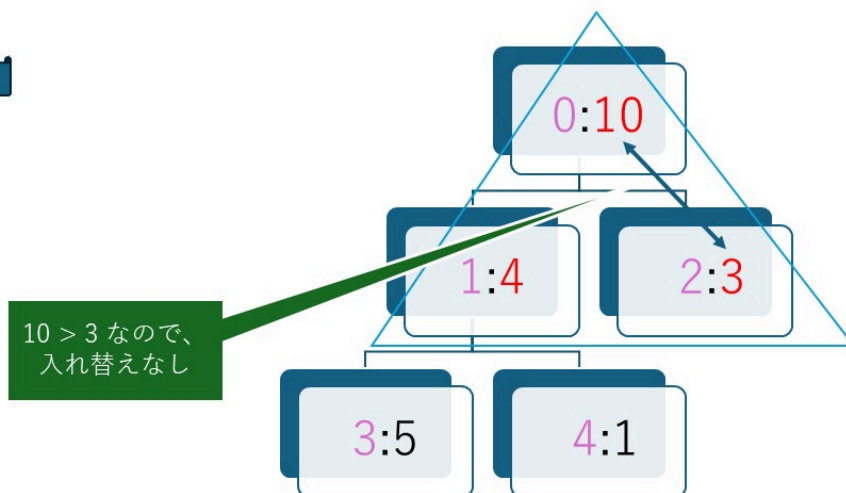
Heap



Array

0	1	2	3	4
10	4	3	5	1

Heap



- 入れ替えが発生したノードの下部を、再ヒープ化します。

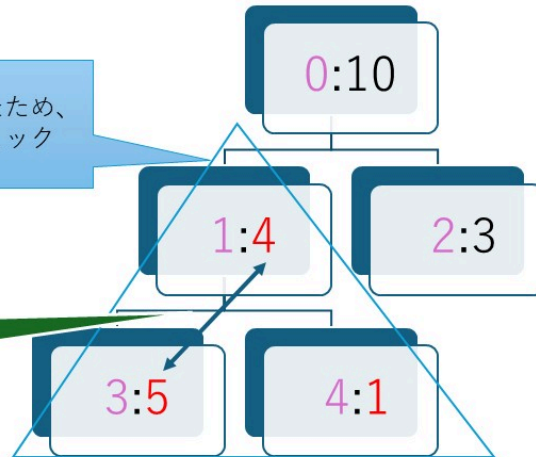
Array

0	1	2	3	4
10	4	3	5	1

Heap

入れ替えが発生したため、
その下部を再チェック

4 < 5 なので、
要入れ替え



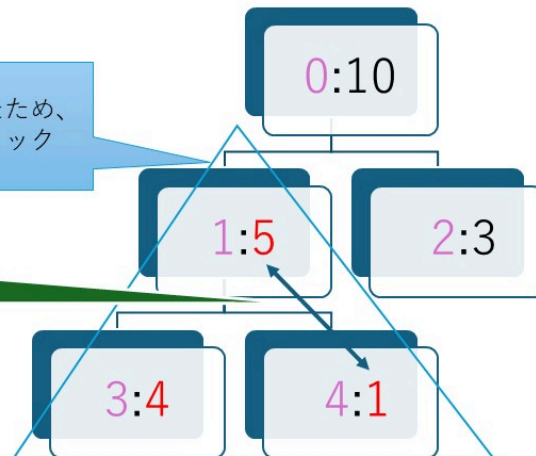
Array

0	1	2	3	4
10	5	3	4	1

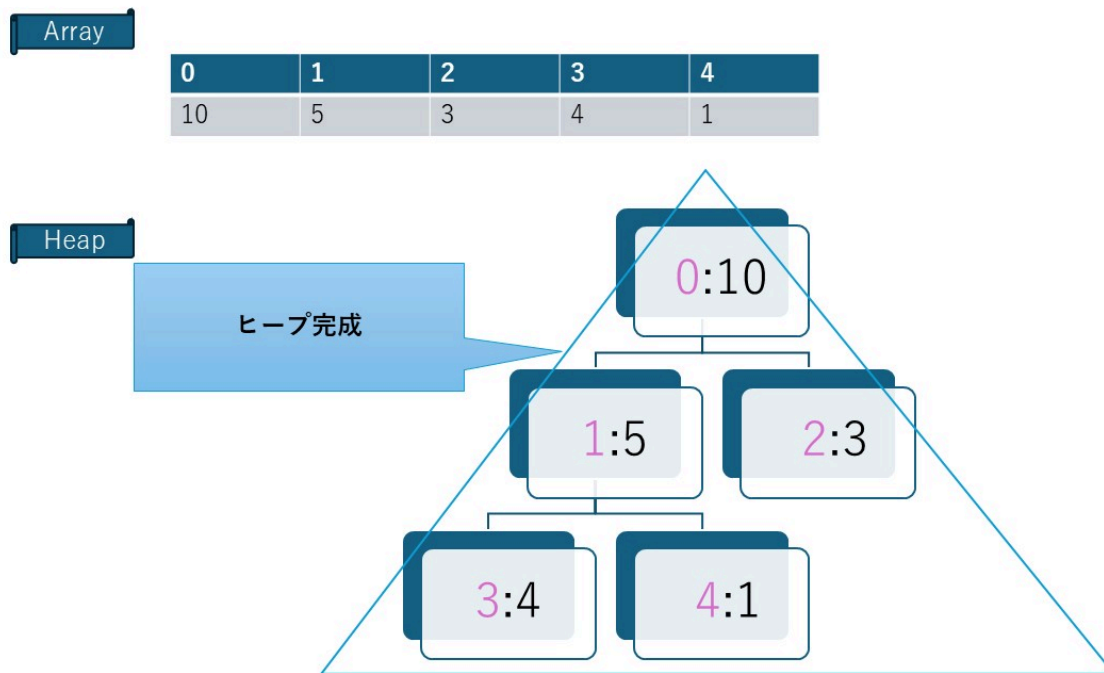
Heap

入れ替えが発生したため、
その下部を再チェック

5 > 1 なので、
入れ替えなし



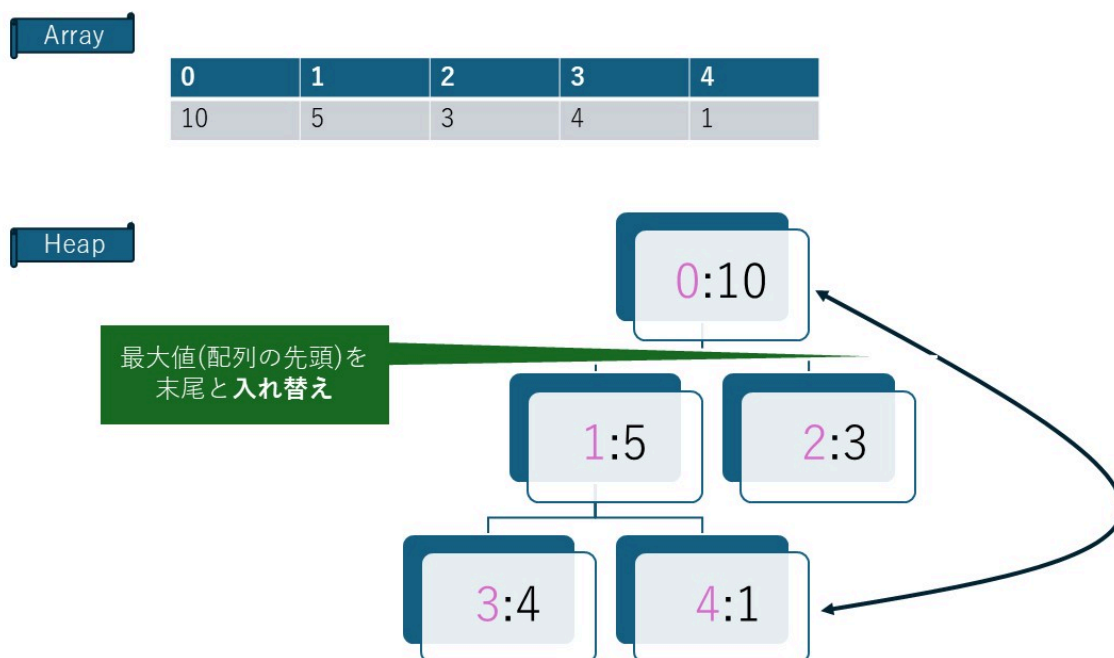
- これで、ヒープ完成!



フェーズ②：最大値を取り出して並べる（決勝・順位決定）

1. 末尾と先頭（最大値）を入れ替え、配列の末尾に置いた最大値を **ソート済み** とします。
2. **作り直し**：残った未ソートのメンバーで、「ヒープ」の形を整えます。
3. **繰り返す**：これを全員分繰り返すと、後ろから順番に大きい順に並んでいきます。

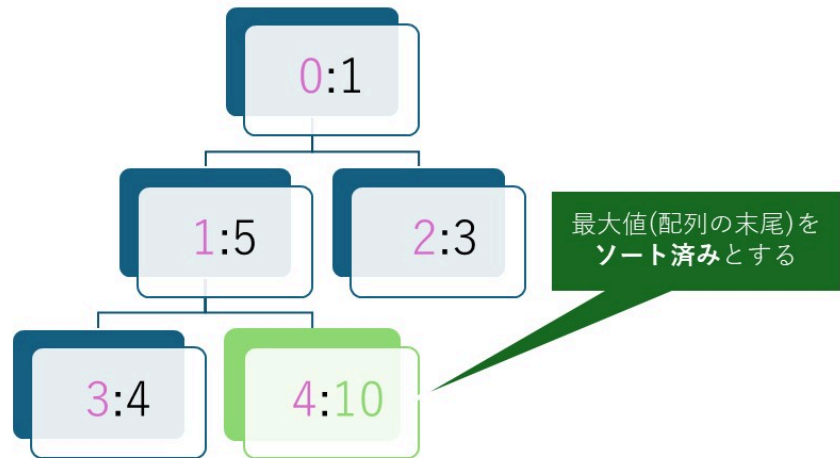
- 先頭の10を末尾と入れ替えて、ソート済みとする。



Array

0	1	2	3	4
1	5	3	4	10

Heap



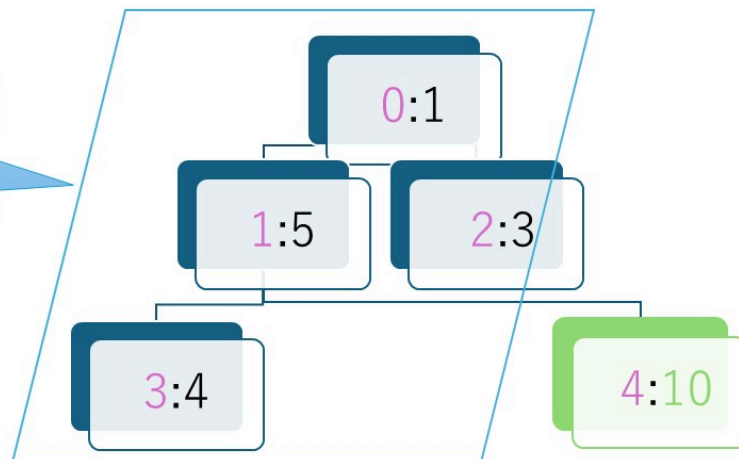
- 残りを再ヒープ化

Array

0	1	2	3	4
1	5	3	4	10

Heap

残りを再ヒープ化

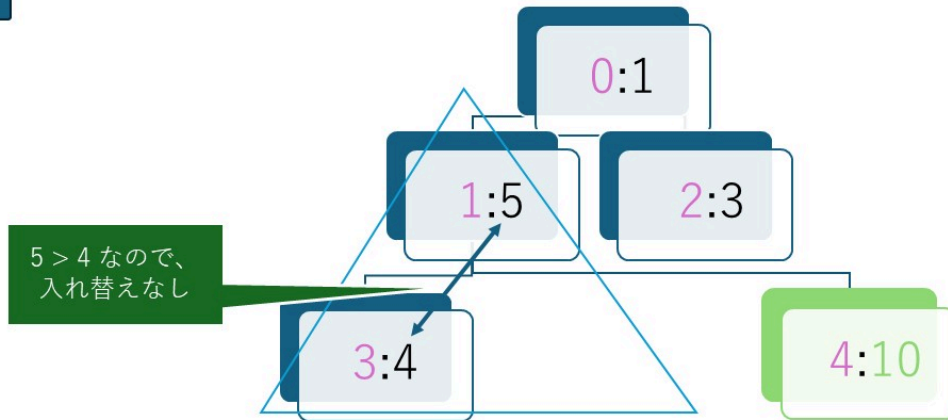


- リーフ (一番下のノード) 以外の親ノードに対して下から順にヒープ化を実行

Array

0	1	2	3	4
1	5	3	4	10

Heap

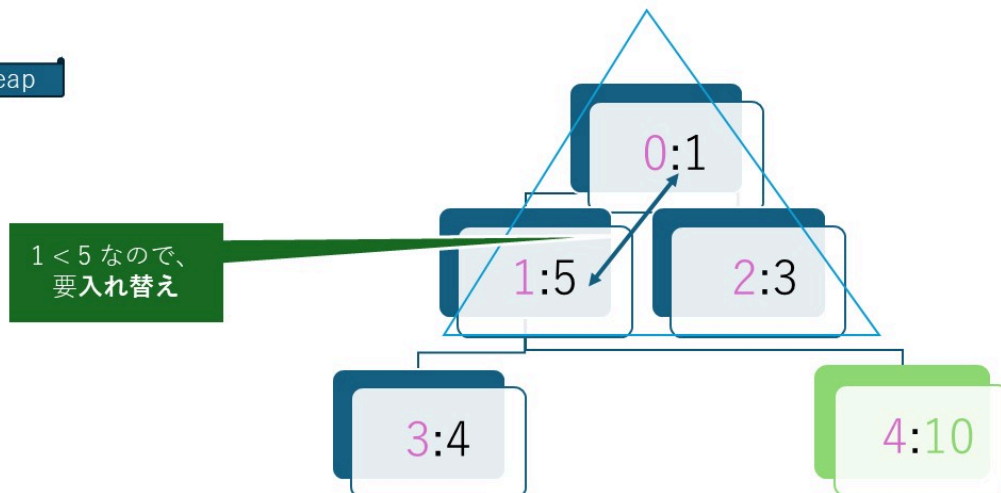


- 最初の分木の再ヒープ化完了

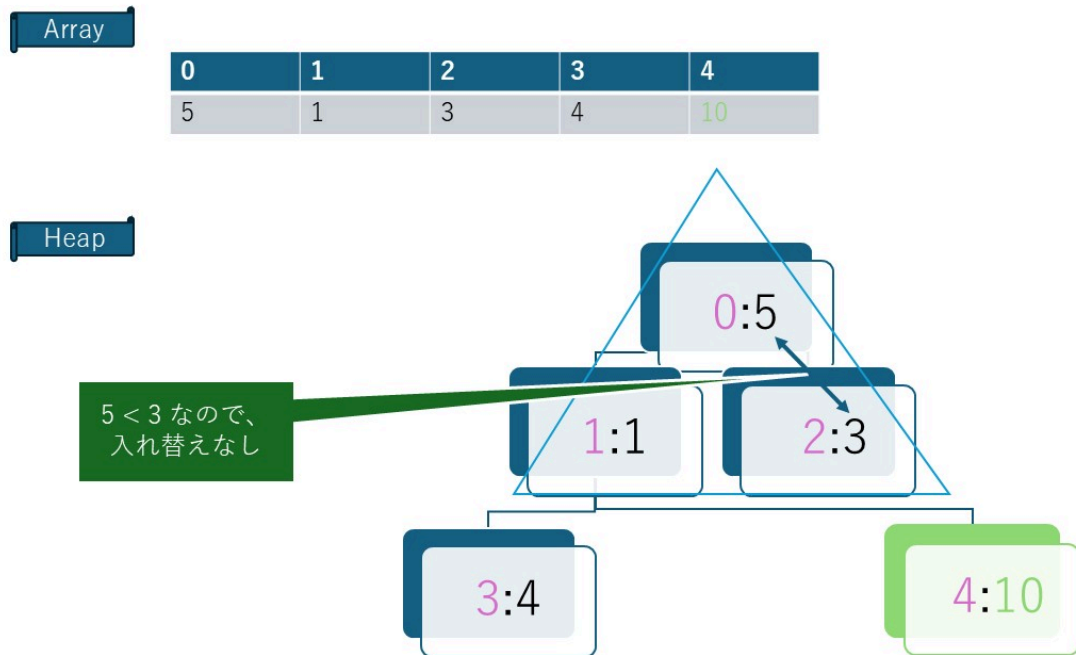
Array

0	1	2	3	4
1	5	3	4	10

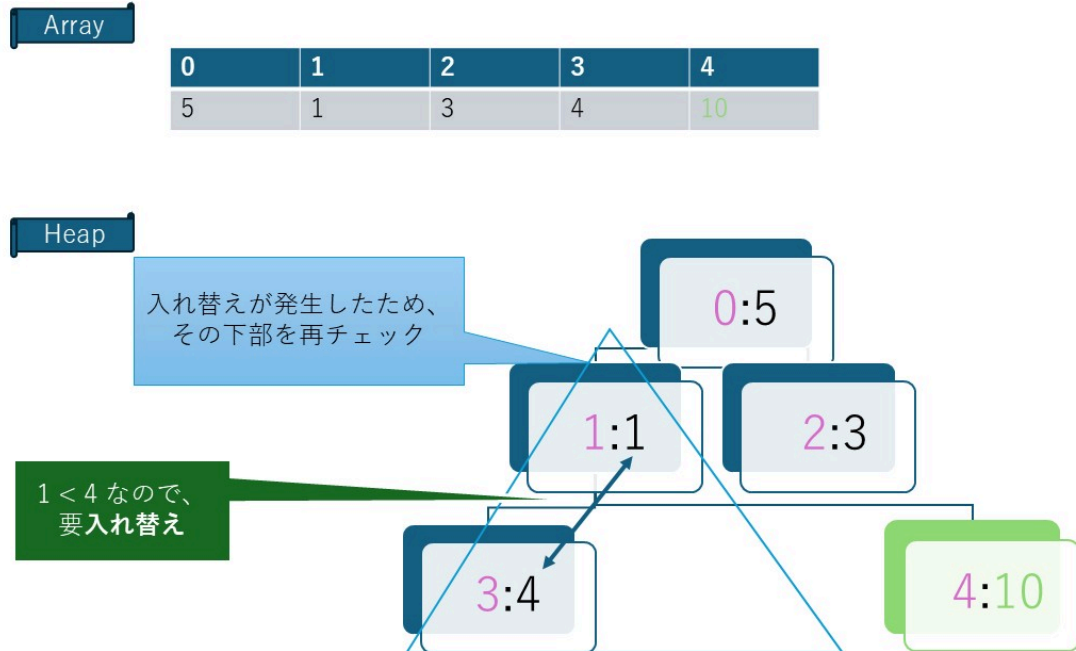
Heap



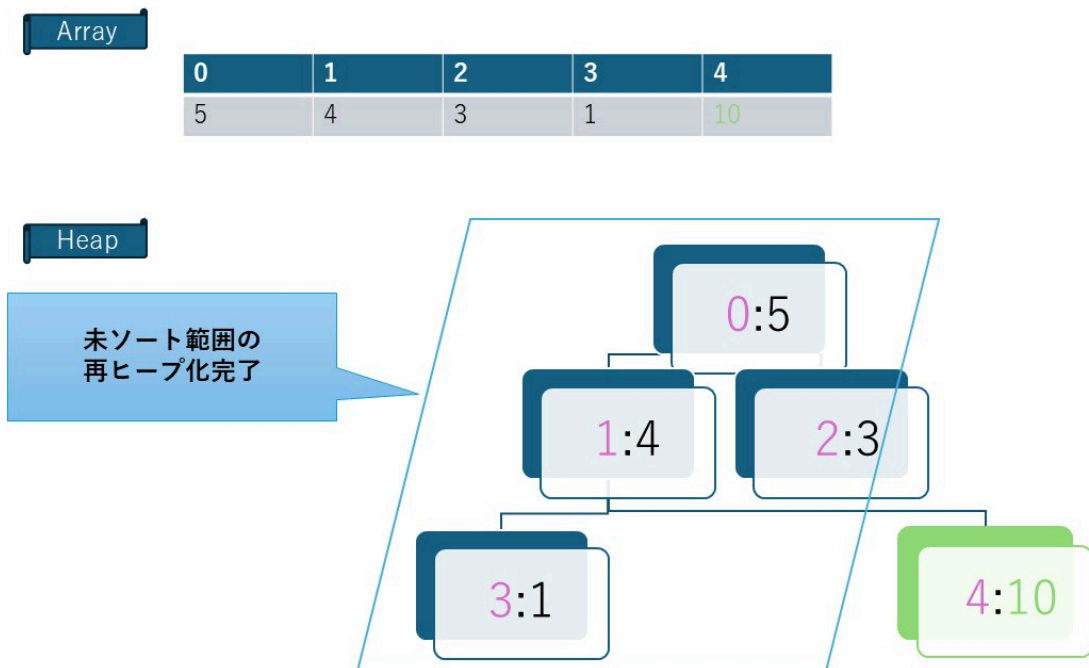
- 次の分木を再ヒープ化



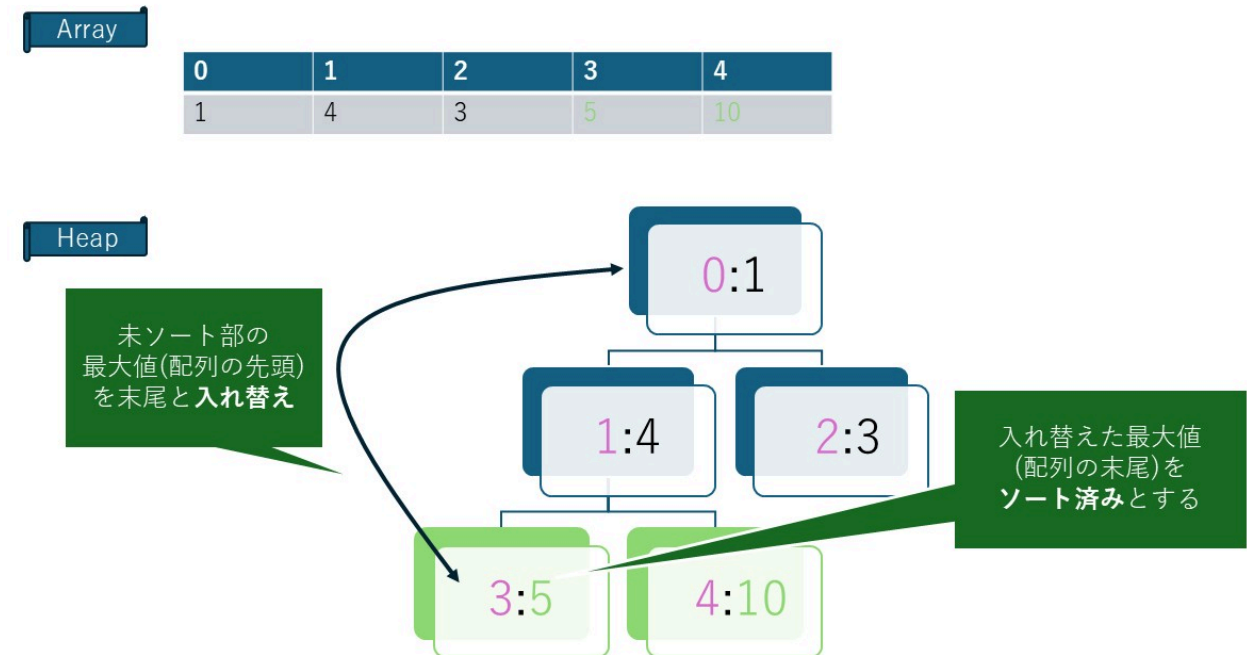
- 入れ替えが発生したので、その下部を再ヒープ化



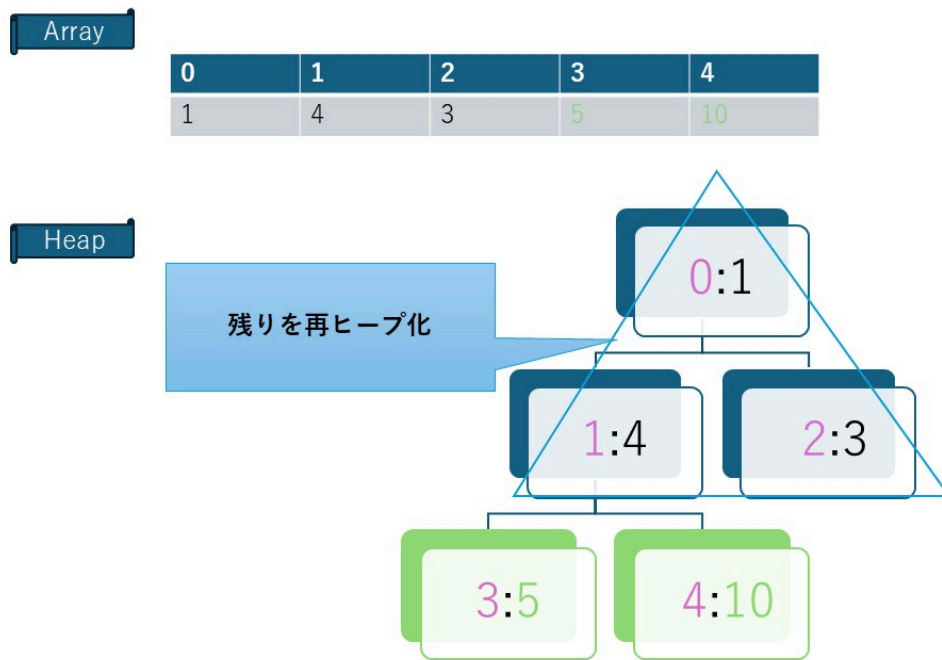
- 未ソート部分の再ヒープ化完了



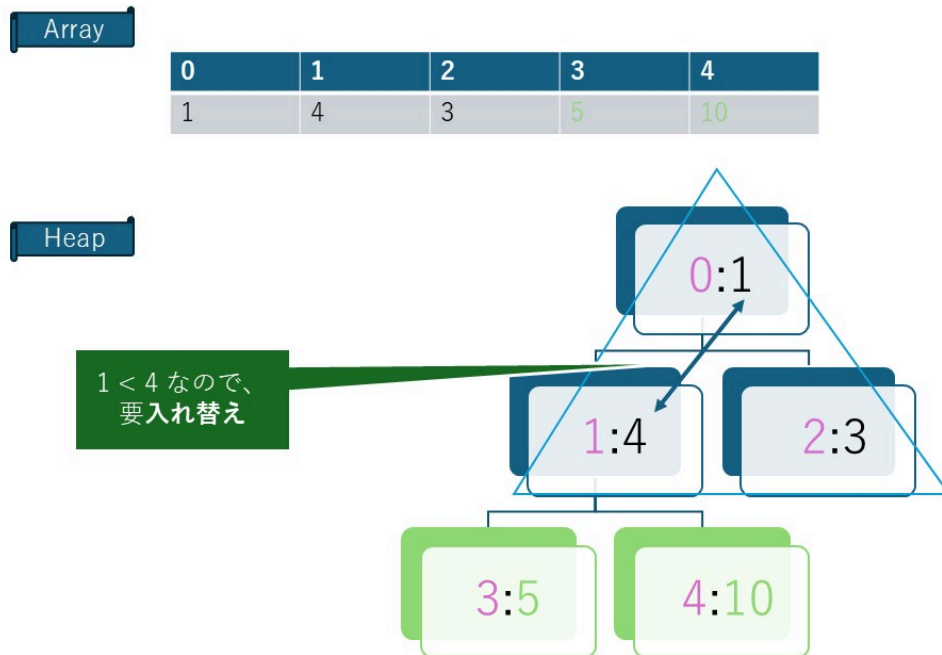
- 先頭の5を末尾と入れ替えて、ソート済みとする。



- 残りを再ヒープ化



- 「親 > 子」のルールに則って、必要な部分を入れ替え

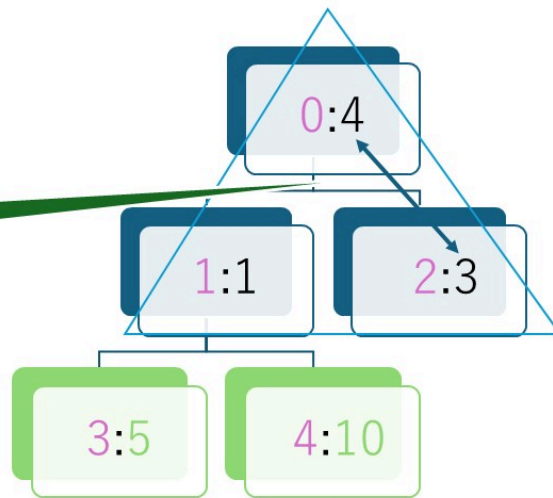


Array

0	1	2	3	4
4	1	3	5	10

Heap

4 > 3 なので、
入れ替えなし



- 既にソート済みの部分には、ノータッチ

Array

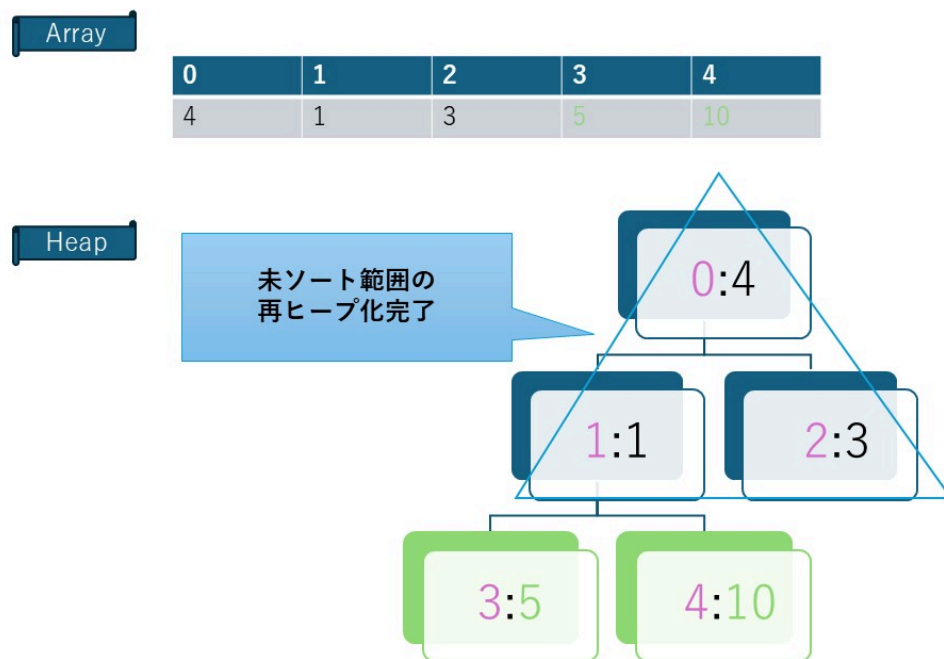
0	1	2	3	4
4	1	3	5	10

Heap

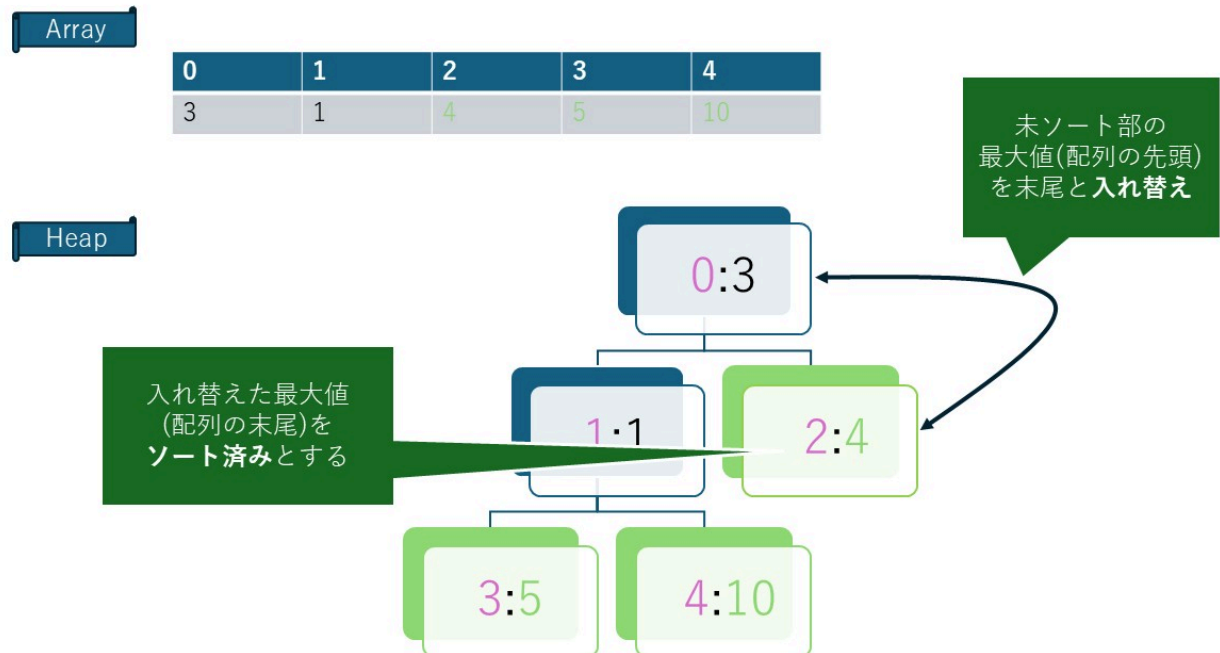
入れ替えたが、
下部はソート済みのため、
再チェックはしない



- 未ソート部分の再ヒープ化完了



- 先頭の4を末尾と入れ替えて、ソート済みとする。



Array

0	1	2	3	4
3	1	4	5	10

Heap

残りを再ヒープ化



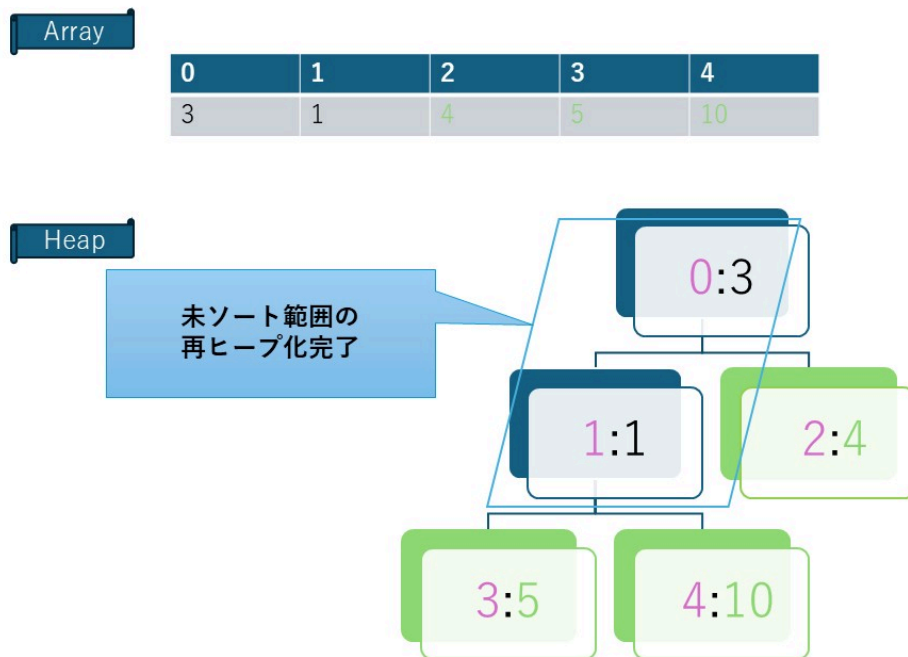
Array

0	1	2	3	4
3	1	4	5	10

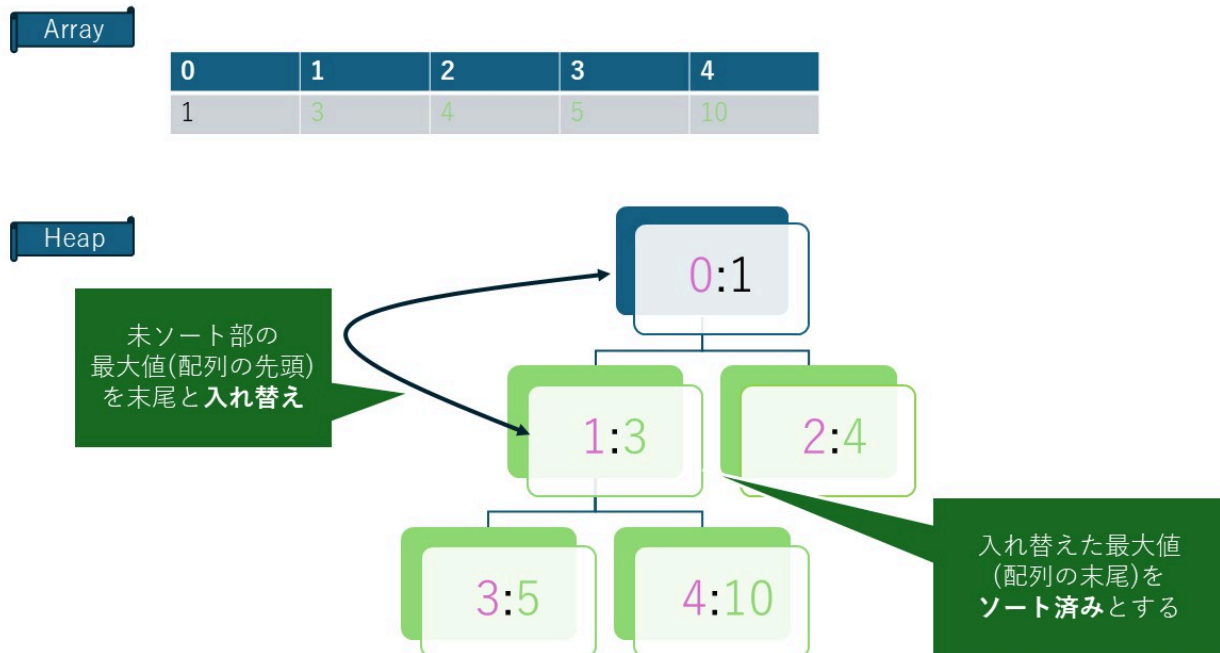
Heap

3 > 1 なので、
入れ替えなし

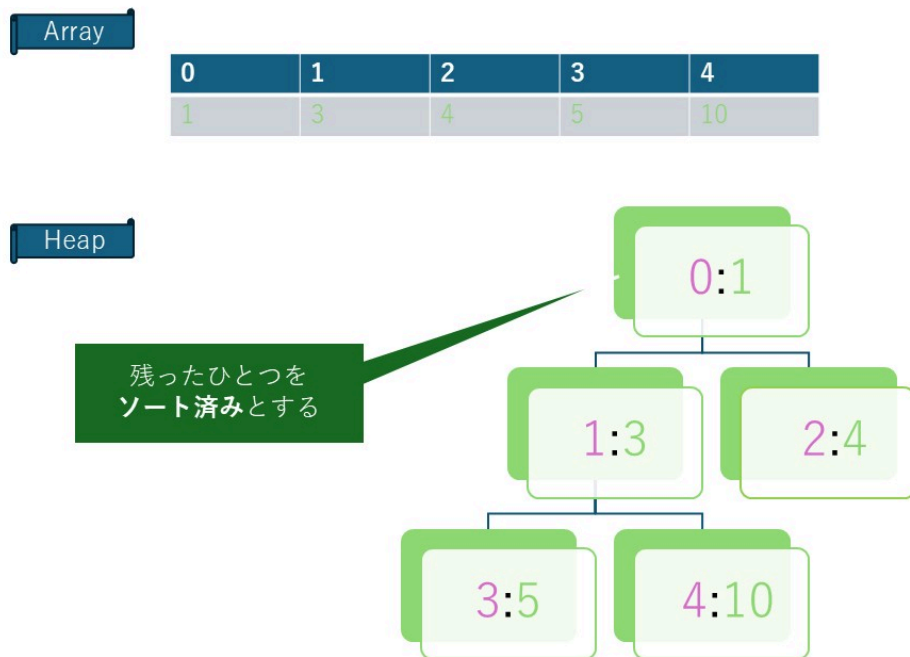
- 未ソート部分の再ヒープ化完了



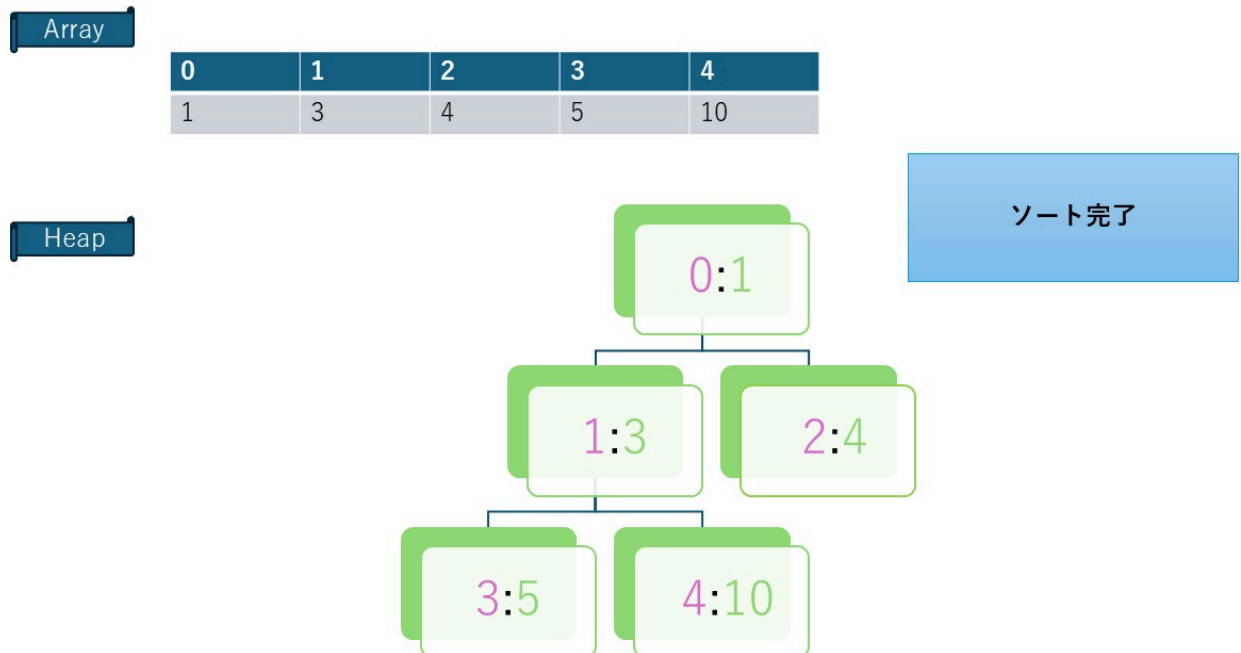
- 先頭の3を末尾と入れ替えて、ソート済みとする。



- 未ソートが先頭のみとなったため、先頭をソート済みとする。



- ソート完了



3. ヒープソートのメリット・デメリット

項目	特徴
安定した速さ	クイックソートのように「データの並びによって極端に遅くなる」ことはありません。
計算量	常に。マージソートと同じく、どんな時でも一定の速さを保ちます。
省スペース	マージソートと違い、 追加の作業スペース（配列）を必要としません 。元の配列の中で完結します。
弱点	構造が少し複雑なため、データ数が少ないときは他のシンプルなソートの方が速いこともあります。

4. 他のソートとの違い

- **選択ソート**は、「全員を端から端まで見て、一番大きい人を探す」という原始的な方法です。
- **ヒープソート**は、「あらかじめトーナメント表（ヒープ）を作っておき、勝ち上がってきた人（頂点）をサッと取り出す」という、**組織的な探し方**です。

5. 実装

ヒープソートは、データを「ヒープ」という特別な木構造（ピラミッドのような形）に整え、その頂点から最大値を取り出していくアルゴリズムです。

C言語で実装する場合、「**ヒープ構造を維持する関数（heapify）**」を作り、それを繰り返し呼び出す形になります。

疑似言語による実装

基本情報技術者試験の最新の記述形式（新形式）に従った疑似言語での実装です。配列のインデックスは1始まりです。

① 要素を入れ替える補助関数（参照渡し）

```
o関数: swap(参照型: 整数型: a, 参照型: 整数型: b)
  整数型: temp ← a
  a ← b
  b ← temp
```

② ヒープ構造を再構築する関数

```

o関数: heapify(整数型: arr[], 整数型: n, 整数型: i)
  整数型: largest ← i
  整数型: left ← 2 * i
  整数型: right ← 2 * i + 1

  if (left ≤ n かつ arr[left] > arr[largest])
    largest ← left
  endif

  if (right ≤ n かつ arr[right] > arr[largest])
    largest ← right
  endif

  if (largest ≠ i)
    swap(arr[i], arr[largest])
    /* 再帰呼び出し */
    heapify(arr, n, largest)
  endif

```

③ ヒープソートの本体関数

```

o関数: heapSort(整数型: arr[], 整数型: n)
  整数型: i

  /* 1. 初期ヒープの構築 */
  for (i を n ÷ 2 の商 から 1 まで 1 ずつ減らす)
    heapify(arr, n, i)
  endfor

  /* 2. 最大値の取り出しと再構成 */
  for (i を n から 2 まで 1 ずつ減らす)
    swap(arr[1], arr[i])
    heapify(arr, i - 1, 1)
  endfor

```

注意点

1. **引数の「参照型」指定 (swap 関数)** 新形式の試験では、呼び出し元の変数を書き換える場合、引数の前に **参照型:** と明示する必要があります。また、**heapify** 内での **swap(arr[i], arr[largest])** は、配列の要素を直接書き換えるため、C言語のようなアドレス演算子 **&** は付けずにそのまま記述します。
2. **初期ヒープ構築のループ開始位置** 「インデックス1始まり」では **n ÷ 2 の商** (切り捨て) になります。
3. **ソート時の境界条件** 最大値を後ろに送るループでは、配列の先頭である **arr[1]** と末尾 **arr[i]** を入れ替えます。その後、未ソート部分の要素数は **i - 1** 個になるため、**heapify(arr, i - 1, 1)** とし

てヒープを再構成します。

C言語によるヒープソートの実装

C言語の場合、インデックスが0始まりになることで、疑似言語のコードから以下の計算規則が変わっている点に注目してください。

- 左の子： $i \times 2 + 1$
- 右の子： $i \times 2 + 2$
- 配列の末尾の要素： $n - 1$

```
#include <stdio.h>

// 二つの値を入れ替える補助関数
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// 木の構造を整える関数（親が子より大きくなるように調整する）
// n: 全体の要素数, i: 根（親）とする位置
void heapify(int arr[], int n, int i) {
    int largest = i;    // とりあえず「親」が最大だと仮定
    int left = 2 * i + 1; // 左の子のインデックス
    int right = 2 * i + 2; // 右の子のインデックス

    // 左の子の方が親より大きければ、最大値を更新
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // 右の子の方が今の最大値より大きければ、最大値を更新
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // もし最大値が親でなければ、入れ替えてさらに下の階層も整える
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        // 入れ替えた後の下の枝（サブツリー）を再帰的に整える
        heapify(arr, n, largest);
    }
}

// ヒープソートの本体
void heapSort(int arr[], int n) {
    // 1. 最初のヒープを作る（予選大会：最大値を頂点に持ってくる）
    // 葉（一番下のノード）以外の親ノードに対して下から順にheapifyを実行
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // 2. 頂点（最大値）を一つずつ取り出し、最後に移動させる（決勝戦）
```

```

    for (int i = n - 1; i > 0; i--) {
        // 頂点（最大値）を現在の最後尾と入れ替える
        swap(&arr[0], &arr[i]);
        // 残った要素（i個）で、再び頂点が最大になるように整える
        heapify(arr, i, 0);
    }
}

int main() {
    int data[] = {4, 10, 3, 5, 1};
    int n = 5;

    printf("ソート前: ");
    for (int i = 0; i < n; i++) printf("%d ", data[i]);
    printf("\n");

    heapSort(data, n);

    printf("ソート後: ");
    for (int i = 0; i < n; i++) printf("%d ", data[i]);
    printf("\n");

    return 0;
}

```

ポイント解説

1. 配列と木構造の対応

- プログラム上は普通の「配列（arr）」ですが、頭の中では「木構造」をイメージしていることを伝えてください。
- インデックス i 番目の要素の子は、必ず $2*i+1$ と $2*i+2$ になるという算術的な法則を利用しています。

2. heapify（ヒープ化）の意味

- これは「3人組（親、左の子、右の子）の中で誰が一番強い（大きい）か？」を決めるミニトーナメントです。親が負けたら場所を交代し、負けた親はさらに下の階層で修行（再帰呼び出し）し直します。

3. ソートの仕上げ

- 「ピラミッドの頂点（最大値）を後ろへ飛ばし、残ったメンバーでまた頂点を決める」という作業を繰り返すことで、配列の後ろから順に大きい数字が埋まっていきます。

資料のまとめ：ソート編の締めくくり

これで主要なソートアルゴリズムの解説と実装がすべて完了しました！